Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

Análisis comparativo de herramientas de simulación mediante un modelo común

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Maria Soledad Buri Camacho Emilio Josue Flores Gallardo Servio Julian Vega Jiménez Yandri Alexander Piscocama Jaramillo
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 y 23 de octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

Objetivos específicos:

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta.

3. Materiales, Reactivos, Equipos y Herramientas

Software y herramientas:

- **Java SE / JDK:** usado para programar la simulación discreta del modelo SIR bajo arquitectura MVC y generar `resultados_java.csv`.

- **NetLogo 7.0.4 (extensión `csv`):** usado exclusivamente para leer el archivo CSV y graficar las curvas del modelo.
- **Visual Studio Code y terminal Konsole (Linux):** edición, compilación y ejecución del código.
- **Git y GitHub:** control de versiones y repositorio del proyecto.

Equipos: computador con procesador de 2 núcleos o superior, 4 GB de RAM y sistema operativo GNU/Linux.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

El modelo epidemiológico **SIR** representa la propagación de una infección en una población cerrada de tamaño constante N, dividida en tres estados:

- S **(Susceptibles):** individuos sanos que pueden contagiarse al tener contacto con un infectado.
- I **(Infectados):** individuos con la enfermedad activa que pueden contagiar a los susceptibles.
- R **(Recuperados):** individuos que superaron la enfermedad y tienen inmunidad permanente.

En todo paso de tiempo t la población total se mantiene constante:

$$N = S(t) + I(t) + R(t) \tag{1}$$

El sistema continuo se describe con las ecuaciones diferenciales:

$$\frac{dS}{dt} = -\beta\frac{SI}{N}, \qquad \frac{dI}{dt} = \beta\frac{SI}{N} - \gamma I, \qquad \frac{dR}{dt} = \gamma I \tag{2}$$

donde β es la tasa de contagio ($S + I \to I + I$) y γ es la tasa de recuperación ($I \to R$). El **número reproductivo básico** (R_0) se calcula como:

$$R_0 = \frac{\beta}{\gamma} = \frac{0.30}{0.10} = 3 \tag{3}$$

Esto significa que, al inicio del brote, un infectado contagia en promedio a 3 personas antes de recuperarse. La Tabla 1 resume los parámetros usados.

Tabla 1: Parámetros de configuración del modelo SIR en Java.

Parámetro	Valor	Función en el modelo
Población total (N)	100	Cantidad total fija de individuos ($S + I + R = 100$).
Infectados iniciales (I_0)	5	Individuos que inician en estado `INFECTADO` en $t = 0$.
Susceptibles iniciales (S_0)	95	Individuos que inician en estado `SUSCEPTIBLE` en $t = 0$.
Recuperados iniciales (R_0)	0	Individuos recuperados al inicio de la simulación.
Tasa de infección (β)	0.30	Probabilidad de contagio en un contacto infectado–susceptible.
Tasa de recuperación (γ)	0.10	Probabilidad de que un infectado pase a recuperado en cada paso.
Tiempo de simulación	100	Pasos discretos simulados ($t = 0$ hasta $t = 100$).
Semilla (`semilla`)	`2026L`	Semilla fija en `Random` para repetir el mismo experimento.

Alcance del grupo: por asignación e indicación del docente, nuestro grupo trabajó únicamente la integración **Java + NetLogo** (sin Python). Toda la simulación se ejecutó en Java exportando los datos a `resultados_java.csv`, y NetLogo se usó solo para leer ese archivo y mostrar las gráficas.

4.2. Las 7 etapas de un proyecto de simulación

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, aplicadas a la implementación del modelo epidemiológico SIR en Java y su visualización mediante NetLogo:

- **Etapa 1. Definición del problema**
  Se planteó la necesidad de simular la propagación de una enfermedad en una población de 100 individuos, con el objetivo de analizar la evolución de susceptibles, infectados y recuperados.
- **Etapa 2. Formulación conceptual**
  Se definieron los tres estados del modelo SIR (S, I y R), las transiciones entre ellos y los parámetros de contagio ($\beta = 0.30$) y recuperación ($\gamma = 0.10$), durante 100 pasos de simulación.
- **Etapa 3. Recolección y análisis de datos**
  Se utilizaron los parámetros establecidos en la guía: población de 100 individuos, 5 infectados iniciales y 95 susceptibles. Estos valores constituyeron las condiciones iniciales del modelo.
- **Etapa 4. Construcción del modelo**
  Se implementó el modelo en Java mediante la arquitectura MVC, utilizando objetos para representar individuos y números pseudoaleatorios para determinar contagios y recuperaciones. Los resultados se almacenaron en un archivo CSV.
- **Etapa 5. Verificación y validación**
  Se comprobó la correcta ejecución del programa, la conservación de la población total ($S + I + R = 100$) y la generación de 101 registros, correspondientes a los tiempos de 0 a 100.
- **Etapa 6. Experimentación y análisis de resultados**
  Se ejecutó la simulación con la semilla `2026L` y se importaron los resultados en NetLogo para observar gráficamente la evolución de susceptibles, infectados y recuperados.
- **Etapa 7. Implementación y documentación**
  Se integraron Java y NetLogo mediante el archivo `resultados_java.csv`. Finalmente, se organizó el proyecto, se documentaron los resultados obtenidos y se elaboraron las conclusiones y recomendaciones de la práctica.

4.3. Arquitectura MVC en Java e integración con NetLogo

El código en Java se organizó con la arquitectura **Modelo–Vista–Controlador (MVC)**, aplicando normas de *Clean Code* y principios **SOLID** para separar el cálculo, la vista en consola y la creación del archivo CSV (Tabla 2):

- **Modelo de datos seguro:** Se creó el enumerado `EstadoSalud` (`SUSCEPTIBLE`, `INFECTADO`, `RECUPERADO`) para evitar errores con cadenas o enteros. La clase `Individuo` guarda el estado de cada persona y `RegistroSir` guarda de forma inmutable (atributos `final`) los conteos de cada paso de tiempo.

- **Actualización por pasos en `ModeloSir`:** Como la guía no fija cuántos contactos ocurren por turno, cada infectado selecciona al azar a un individuo distinto por paso. Para no mezclar estados dentro del mismo turno, se usan dos arreglos (`actuales` y `siguientes`). Los contagios (probabilidad $\beta = 0.30$) y recuperaciones (probabilidad $\gamma = 0.10$) se evalúan sobre `actuales` y se guardan en `siguientes`. Así, un recién contagiado solo puede contagiar o recuperarse a partir del siguiente paso.
- **Responsabilidad única (SRP):** `ModeloSir` solo ejecuta la simulación, `ExportadorCsv` escribe el archivo en UTF-8 y `VistaConsola` imprime los datos en pantalla. `ControladorSir` recibe estas tres clases desde `Main.java` y coordina la ejecución.
- **Lectura en NetLogo (`Grafica.nlogox`):** Con la extensión `csv`, el botón `setup` abre `resultados_java.csv`, salta el encabezado, guarda las 101 filas en la lista `datos-simulacion` y llama a `graficar-datos`, que usa `plotxy` para trazar la gráfica general y las tres gráficas individuales.

Tabla 2: Estructura modular del proyecto APE 004 (Java MVC + NetLogo).

Archivo / Clase	**Responsabilidad en el proyecto**
`modelo/EstadoSalud.java`	Define el `enum` con los estados `SUSCEPTIBLE`, `INFECTADO` y `RECUPERADO`.
`modelo/Individuo.java`	Representa a cada individuo y su estado de salud actual.
`modelo/RegistroSir.java`	Objeto inmutable que guarda los datos de un paso (`tiempo`, S, I, R).
`modelo/ModeloSir.java`	Simula de $t = 0$ a 100 con copias de estado y valida que $S + I + R = 100$.
`modelo/ExportadorCsv.java`	Genera el archivo `resultados_java.csv` con las 101 filas de datos.
`vista/VistaConsola.java`	Imprime en consola los parámetros, la tabla cada 10 pasos y la ruta del CSV.
`controlador/ControladorSir.java`	Conecta el modelo, la vista de consola y el exportador CSV.
`Main.java`	Punto de entrada que configura el modelo con la semilla `2026L`.
`Grafica.nlogox`	Lee `resultados_java.csv` en NetLogo y dibuja las 4 gráficas.

5. Resultados

5.1. Ejecución en Java y salida en consola

Al compilar y correr el programa en la terminal (`javac Main.java` y `java Main.java`), se ejecuta la simulación de 100 pasos, se muestra el resumen cada 10 pasos y se crea `resultados_java.csv` (Figura 1).

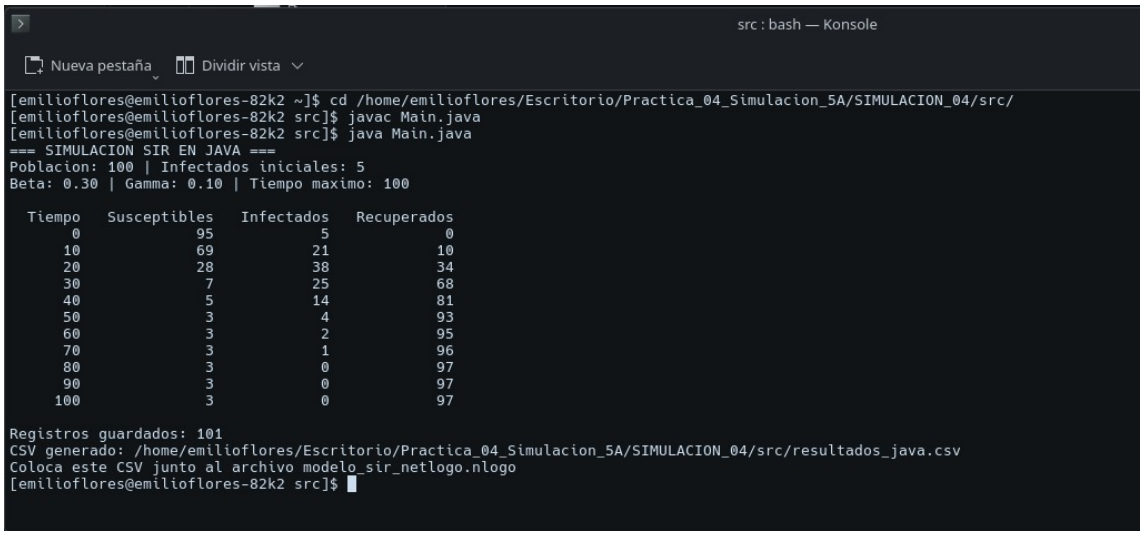


Figura 1: Compilación y ejecución del modelo SIR en Java mostrando la tabla resumen y la creación de `resultados_java.csv`.

La Tabla 3 resume los datos impresos cada 10 pasos junto con los puntos clave del archivo `resultados_java.csv`: los picos máximos de contagio ($t = 24$ y $t = 26$ con 44 infectados) y el fin del brote ($t = 73$ con $I = 0$).

Tabla 3: Evolución de Susceptibles (S), Infectados (I) y Recuperados (R) en Java (`semilla = 2026L`).

Tiempo (t)	**Susceptibles (S)**	**Infectados (I)**	**Recuperados (R)**	**Total (N)**	**Observación del estado**
0	95	5	0	100	Condición inicial de la simulación.
10	69	21	10	100	Aumento rápido de contagios ($R_0 = 3$).
20	28	38	34	100	Los recuperados superan a los susceptibles.
24 (Hito)	16	44	40	100	Primer pico máximo de infectados ($I_{\text{máx}} = 44$).
26 (Hito)	8	44	48	100	Segundo pico máximo de infectados ($I_{\text{máx}} = 44$).
30	7	25	68	100	Caída rápida de los casos activos.
40	5	14	81	100	Más del 80 % de la población ya es inmune.
50	3	4	93	100	Quedan solo 3 susceptibles sin contagiar.
60	3	2	95	100	Recuperación de los últimos infectados.
70	3	1	96	100	Queda un solo individuo infectado.
73 (Hito)	3	0	97	100	Fin del brote ($I = 0$); el sistema se estabiliza.
80	3	0	97	100	Estado estable sin nuevos contagios.
90	3	0	97	100	Estado estable mantenido.
100	3	0	97	100	Fin de la simulación en el paso máximo.

5.2. Visualización gráfica de resultados en NetLogo

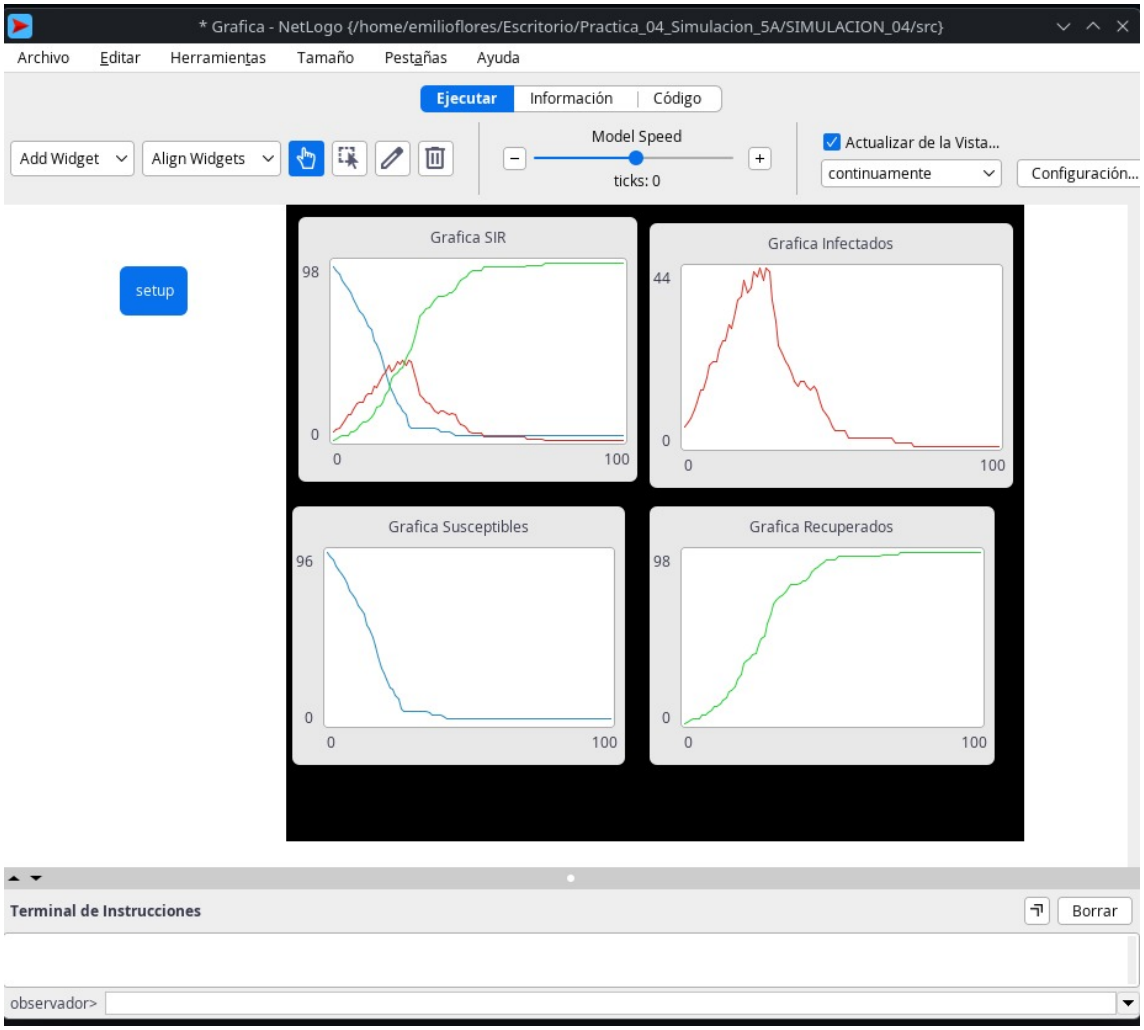


Figura 2: Interfaz de NetLogo (`Grafica.nlogox`) mostrando la carga del CSV y las cuatro gráficas del modelo SIR.

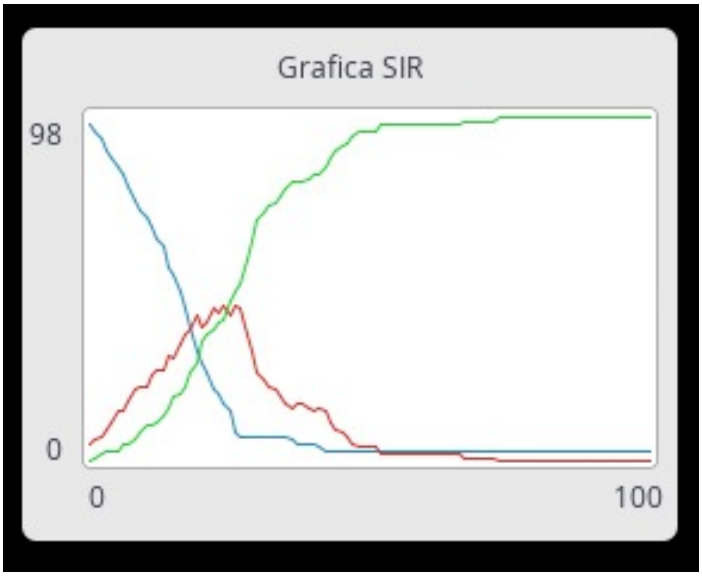


Figura 3: Comparación gráfica de los modelos.



Figura 4: Gráfica Infectados.

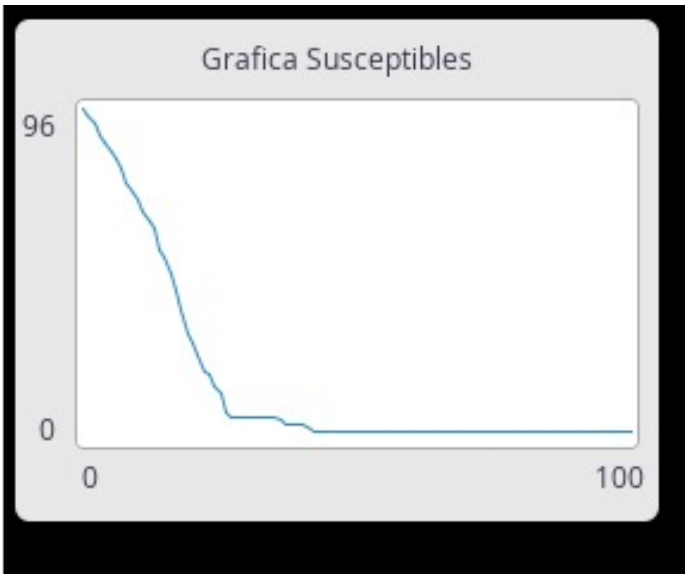


Figura 5: Gráfica Susceptibles.



Figura 6: Gráfica Recuperados.

5.3. Análisis de resultados y números aleatorios

- **Comportamiento de las curvas (S, I, R):** De $t = 0$ a $t = 26$, los susceptibles bajan de 95 a 8 porque al inicio casi todos los contactos son con gente sana y la probabilidad de contagio ($\beta = 0.30$) es el triple que la de recuperación ($\gamma = 0.10$). Esto lleva la curva de infectados a su máximo de 44 casos en $t = 24$ y $t = 26$. Desde $t = 27$, al quedar pocos susceptibles ($S \leq 7$), casi todos los contactos ocurren con personas ya infectadas o recuperadas, haciendo que los infectados bajen hasta 0 en $t = 73$.
- **Fin del brote sin contagio total:** La simulación termina con 97 recuperados y 3 susceptibles que nunca se enfermaron. Esto ocurre porque, al haber tantos recuperados inmunes, los últimos infectados se recuperan antes de toparse por azar con los pocos susceptibles restantes.
- **Diferencias por números aleatorios:** Aunque se usen los mismos parámetros ($N = 100, I_0 = 5, \beta = 0.30, \gamma = 0.10$) en Java, Python y NetLogo, cada lenguaje tiene un generador aleatorio distinto (`java.util.Random` frente a Mersenne Twister). Como en cada paso se sortea a quién se contacta y si hay contagio o recuperación, la secuencia aleatoria cambia ligeramente el paso exacto del pico, pero mantiene la misma tendencia general del modelo SIR.

5.4. Comparación práctica de las herramientas de simulación

En la Tabla 4 se comparan las tres herramientas según la práctica realizada.

Tabla 4: Comparación práctica entre NetLogo, Python/SimPy y Java.

Característica	**NetLogo**	**Python / SimPy**	**Java**
Facilidad de uso	Alta para modelos visuales; sintaxis corta pensada para agentes.	Alta; código corto y fácil de leer con librerías científicas.	Media; pide definir clases, tipos de datos y paquetes.
Manejo de agentes	Nativo; las tortugas ya tienen posición, estado y vista en 2D.	Mediante funciones generadoras (`yield`) o clases propias.	Orientado a objetos con clases (`Individuo`) y listas tipadas.
Generación de datos	Rápida en pantalla, pero menos cómoda para exportar grandes lotes.	Muy sencilla con listas, `pandas` o el módulo `csv`.	Segura y ordenada usando `BufferedWriter` y `RegistroSir`.
Visualización	Excelente; trae gráficas (`plotxy`) y botones listos en la interfaz.	Usa librerías externas como `Matplotlib` o páginas con `Flask`.	Sin gráficas nativas directas; conviene exportar CSV a otra app.
Flexibilidad	Ideal para sistemas de agentes (ABM) y demostraciones visuales.	Muy alta para unir simulación, estadística y ciencia de datos.	Muy alta para aplicar arquitectura MVC, SOLID y proyectos grandes.
Tiempo de ejecución	Moderado si dibuja la interfaz en cada paso (*tick*).	Rápido con vectores, pero más lento en bucles largos puros.	Muy rápido; la JVM ejecuta los 100 pasos casi al instante.
Integración	Buena usando extensiones como `csv` o `pyNetLogo`.	Muy alta con archivos CSV, bases de datos y APIs.	Directa y portable mediante archivos CSV, JSON o SQL.

5.5. Cumplimiento de resultados esperados

Tabla 5: Verificación de los resultados esperados en la práctica APE 004.

Resultado esperado	Cumplimiento en la práctica
Modelo SIR implementado en Java.	Programado con arquitectura MVC y semilla `2026L`.
Archivo CSV generado por la simulación.	Creación de `resultados_java.csv` con 101 filas ($t = 0$ a 100).
Visualización en NetLogo desde el CSV.	Carga con `extensions [csv]` en `Grafica.nlogox`.
Gráficas de susceptibles, infectados y recuperados.	Cuatro gráficas generadas en NetLogo (general e individuales).
Análisis de herramientas y números aleatorios.	Explicado en las secciones 5.3 y 5.4 (Tabla 4).

6. Preguntas de Control

1. **¿Qué representa el modelo SIR?**
   Es un modelo epidemiológico que muestra cómo avanza una enfermedad infecciosa en una población cerrada, dividiendo a las personas en Susceptibles (S), Infectados (I) y Recuperados (R).

2. **¿Qué representan los estados S, I y R?**
   - S **(Susceptibles):** individuos sanos que pueden contagiarse.
   - I **(Infectados):** individuos enfermos que pueden contagiar a los susceptibles.
   - R **(Recuperados):** individuos que ya sanaron y tienen inmunidad permanente.

3. **¿Qué función cumplen los parámetros β y γ?**
   β (tasa de infección) es la probabilidad de que un contacto entre un infectado y un susceptible produzca contagio; γ (tasa de recuperación) es la probabilidad de que un infectado se recupere en cada paso de tiempo.

4. **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?**
   Para que la comparación sea justa. Si se mantienen fijos $N = 100, I_0 = 5, \beta = 0.30, \gamma = 0.10$ y $T = 100$, las diferencias observadas se deben solo a cada herramienta y no a un cambio en los datos de entrada.

5. **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?**
   Porque la simulación usa probabilidades en cada paso y cada lenguaje tiene su propio generador de números pseudoaleatorios. Eso cambia los sorteos de contactos y contagios entre plataformas aunque sigan la misma tendencia.

6. **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**
   Trae integrados el mundo de agentes (*turtles*), los controles y las gráficas en una sola ventana, lo que permite ver el comportamiento del modelo escribiendo muy poco código.

7. **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**
   Permite programar colas, esperas y recursos compartidos de forma directa usando generadores de Python (`yield`), avanzando de un evento a otro sin recorrer pasos vacíos.

8. **¿Qué ventajas ofrece Java para implementar modelos de simulación?**
   Ofrece alta velocidad de ejecución, tipado estático que evita errores al compilar y facilidad para ordenar el código con clases, arquitectura MVC y principios SOLID.
9. **¿Qué función cumple el archivo CSV en la integración Python/Java $\rightarrow$ NetLogo?**
   Sirve como puente de datos: guarda en texto plano las 101 filas calculadas por Java para que NetLogo las lea con la extensión `csv` y las grafique sin recalcular la simulación.
10. **¿Qué ventajas presenta separar la generación de datos de la visualización?**
    Permite correr los cálculos rápido en consola con Java, dejar guardados los resultados en el archivo CSV para revisarlos cuando sea necesario y usar NetLogo únicamente para armar las gráficas.

7. Conclusiones

- Se implementó el modelo SIR en Java con arquitectura MVC, comprobando en los 101 registros ($t = 0$ a 100) que la población total se mantiene en 100 individuos y que separar los arreglos de estado evita contagios y recuperaciones en el mismo turno.
- La conexión entre Java y NetLogo mediante `resultados_java.csv` funcionó de forma limpia y rápida: Java resolvió toda la simulación en consola y NetLogo trazó la gráfica general y las tres curvas individuales sin repetir cálculos.
- Con la semilla 2026L y $R_0 = 3$, el brote subió rápido hasta un pico de 44 infectados en $t = 24$ y $t = 26$, y luego bajó por falta de susceptibles hasta terminar en $t = 73$ con 97 recuperados y 3 susceptibles sanos.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 004: Análisis comparativo de herramientas de simulación mediante un modelo común*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 4: Herramientas de Simulación y Modelo Epidemiológico SIR*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. W. O. Kermack y A. G. McKendrick, “A contribution to the mathematical theory of epidemics”, *Proceedings of the Royal Society of London. Series A*, vol. 115, no. 772, pp. 700–721, 1927.
4. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
5. U. Wilensky, *NetLogo 7.0.4 User Manual*. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL. Disponible en: `https://ccl.northwestern.edu/netlogo/`
6. Oracle Corporation, *Java Platform, Standard Edition (Java SE) Documentation*. Disponible en: `https://docs.oracle.com/en/java/`

9. Anexos

Anexo A. Compilación en Java e integración con NetLogo

Repositorio del proyecto: Repositorio GitHub

```
# 1. Clonar el repositorio y entrar a la carpeta del proyecto
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git
cd Repositorio-Simulacion/Practica_04_Simulacion_5A/SIMULACION_04/src

# 2. Compilar y ejecutar la simulacion SIR en Java
javac Main.java
java Main.java

# 3. Verificar la creacion de resultados_java.csv (101 registros)
# 4. Abrir Grafica.nlogox en NetLogo y presionar el boton 'setup'
```

Anexo B. Declaración de uso de inteligencia artificial

Siguiendo los principios de transparencia y honestidad de la asignatura, declaro que durante el desarrollo de esta práctica utilicé herramientas de inteligencia artificial generativa como apoyo para estructurar la arquitectura MVC del proyecto en Java y programar el script de lectura y graficación en NetLogo con la extensión `csv`.

Prompt utilizado para el desarrollo de la arquitectura y diseño del código:

> *"Partiendo del pseudocódigo cargado en la guía de la práctica para el modelo epidemiológico SIR discreto ($N = 100$, infectados iniciales $= 5$, $\beta = 0.30$, $\gamma = 0.10$, tiempo máximo $= 100$), implementa la simulación exclusivamente en Java aplicando una arquitectura Modelo-Vista-Controlador (MVC) y diseño orientado a objetos que respete las normas de Clean Code y los principios SOLID. Modela los estados mediante un enum (`EstadoSalud`), encapsula a cada agente en la clase `Individuo` y los resultados por paso en una clase inmutable (`RegistroSir`), realiza la transición de estados en `ModeloSir` usando copias del arreglo para que los nuevos contagios no actúen en el mismo paso, delega la escritura del archivo `resultados_java.csv` a una clase dedicada (`ExportadorCsv`) y muestra el resumen en consola mediante `VistaConsola`. Adicionalmente, implementa el código en NetLogo usando la extensión `csv` para importar `resultados_java.csv` sin volver a simular los datos y generar cuatro gráficas: la gráfica general SIR y las gráficas individuales de susceptibles, infectados y recuperados."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)
- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)

- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto y NotebookLM**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.

Anexo C. link del video explicativo

https://youtu.be/L8biA2CSSU0